

神经网络常用激活函数

激活函数是神经网络中引入非线性变换的关键组件，使网络能够学习复杂的模式。以下是几种最常用的激活函数：

1. Sigmoid

- **公式:** $f(x) = \frac{1}{1+e^{-x}}$
- **输出范围:** (0, 1)
- **特点:** 将输入映射到 0~1 之间，适合二分类输出层
- **缺点:** 易导致梯度消失，输出非零中心，计算指数开销大

2. Tanh (双曲正切)

- **公式:** $f(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$
- **输出范围:** (-1, 1)
- **特点:** 零中心输出，比 Sigmoid 梯度更强
- **缺点:** 仍存在梯度消失问题

3. ReLU (修正线性单元)

- **公式:** $f(x) = \max(0, x)$
- **输出范围:** $[0, \infty)$
- **特点:** 计算简单，收敛快，能有效缓解梯度消失
- **缺点:** 部分神经元可能"死亡" (Dead ReLU)

4. Leaky ReLU

- **公式:** $f(x) = \max(\alpha x, x)$, 通常 $\alpha = 0.01$
- **特点:** 解决 Dead ReLU 问题，负区间有微小斜率

5. Softmax

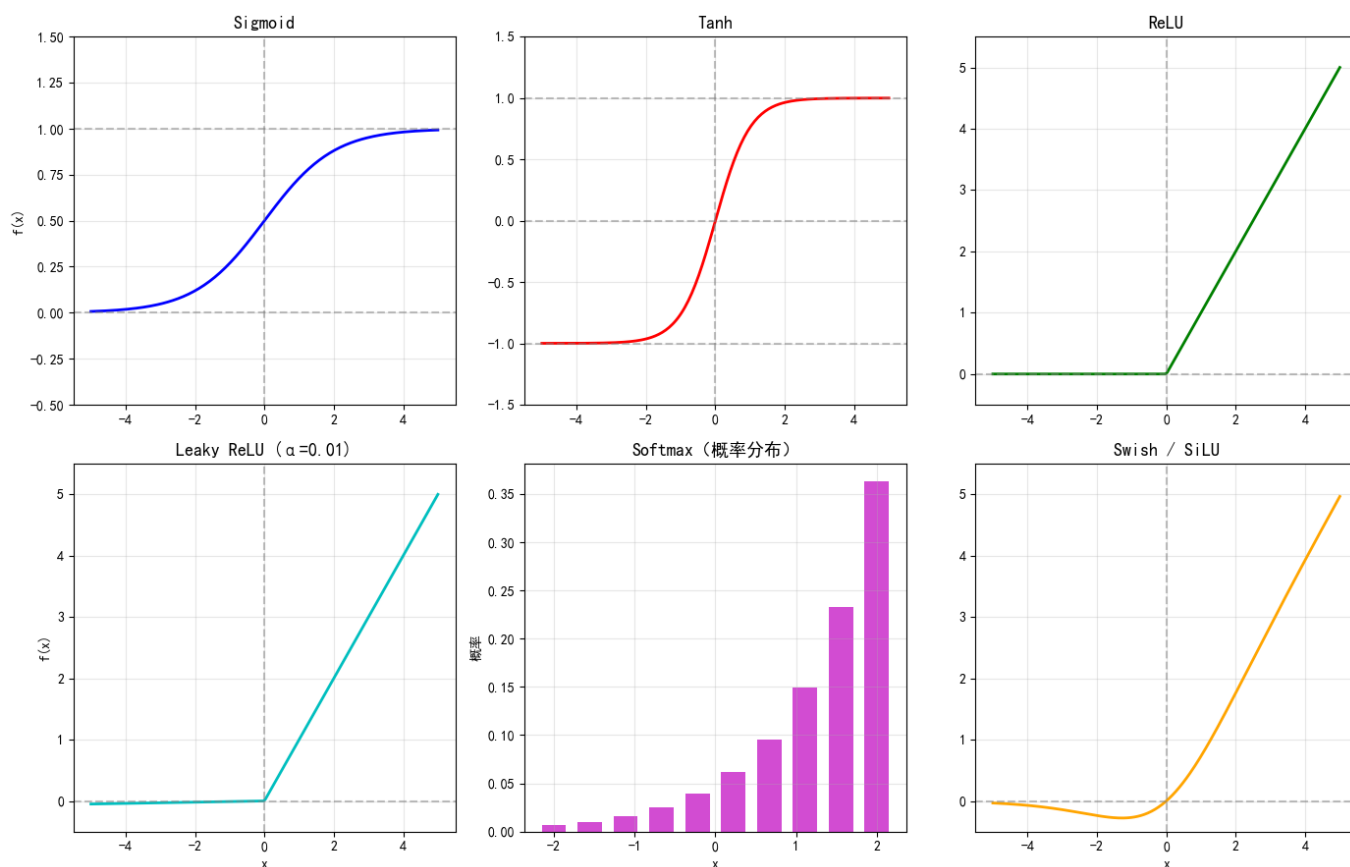
- **公式:** $f(x_i) = \frac{e^{x_i}}{\sum_j e^{x_j}}$

- **输出范围:** (0, 1), 且所有输出和为 1
- **特点:** 多分类输出层标配, 将 logits 转换为概率分布

6. Swish / SiLU

- **公式:** $f(x) = x \cdot \sigma(x) = \frac{x}{1+e^{-x}}$
- **特点:** 谷歌提出, 无上界有下界, 平滑非单调

下面用代码绘制这些激活函数的图像:



欠拟合、拟合与过拟合

1. 基本概念

在机器学习和神经网络训练中, 模型在训练数据上的表现可以分为三种情况:

2. 欠拟合 (Underfitting)

- **定义:** 模型未能充分学习训练数据的特征, 在训练集和测试集上表现都很差

- **表现:** 训练误差高, 测试误差也高

- **原因:**

- 模型过于简单 (层数太少、神经元太少)
- 训练不充分 (迭代次数不够)
- 特征提取不足

- **解决办法:**

- 增加模型复杂度 (加深网络、增加神经元)
 - 增加训练轮数
 - 进行特征工程, 提取更有用的特征
 - 减少正则化强度
-

3. 拟合 (Fitting / Good Fit)

- **定义:** 模型很好地学到了训练数据中的有效模式, 并且在未见过的数据上也有良好的泛化能力
 - **表现:** 训练误差和测试误差都较低, 且两者差距不大
 - **特点:** 这是模型训练的**理想状态**, 既学到了数据中的真实规律, 又没有过度学习噪声
-

4. 过拟合 (Overfitting)

- **定义:** 模型过度学习了训练数据中的细节和噪声, 导致在训练集上表现极好, 但在测试集 (新数据) 上表现很差
- **表现:** 训练误差极低, 但测试误差高, **泛化能力差**
- **原因:**
 - 模型过于复杂 (参数过多), "死记硬背"了训练数据
 - 训练数据太少或缺乏代表性
 - 训练时间过长
 - 数据中存在噪声, 模型将噪声也学习了进去

5. 解决过拟合的常用方法

5.1 增加训练数据量 (Data Augmentation)

- 收集更多数据，或通过数据增强（旋转、裁剪、翻转等）扩充数据集
- 数据越多，模型越难“死记硬背”，被迫学习通用规律

5.2 降低模型复杂度

- 减少网络层数或每层神经元数量
- 使用更简单的模型架构

5.3 正则化 (Regularization)

- **L1 正则化 (Lasso)**：在损失函数中加入权重绝对值之和，产生稀疏解
- **L2 正则化 (Ridge / Weight Decay)**：在损失函数中加入权重平方和，限制权重过大

5.4 Dropout

- 训练时随机丢弃一部分神经元（如 50%），迫使网络不依赖单一特征
- 相当于每次训练一个不同的子网络，最终起到集成学习的效果

5.5 早停法 (Early Stopping)

- 监控验证集误差，当验证集误差不再下降时提前终止训练
- 防止模型在训练集上过度优化

5.6 Batch Normalization

- 对每层的输入进行归一化，稳定训练过程
- 本身具有一定的正则化效果，可以缓解过拟合

5.7 交叉验证 (Cross-Validation)

- 将数据分成多份，轮流用其中一份做验证，其余做训练
 - 更充分地利用有限数据评估模型泛化能力
-

6. 直观理解

| 状态 | 训练误差 | 测试误差 | 泛化能力 |

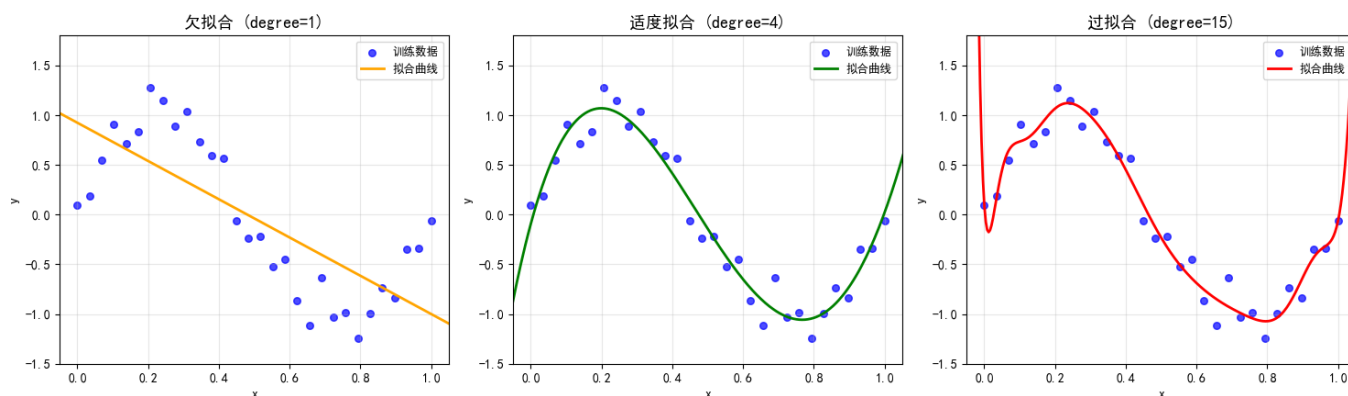
| :---: | :-----: | :-----: | :-----: |

| 欠拟合 | 高 | 高 | 差 |

| 适度拟合 | 较低 | 较低 | 好 |

| 过拟合 | 极低 | 高 | 差 |

下面用代码演示过拟合与正则化的效果：



3D 数据 (h, w, c) 转 4D 数据 (n, c, h, w)

在图像处理和深度学习中，一张彩色图片通常表示为 3 维数组 (H, W, C)：

- H: 高度 (Height)
- W: 宽度 (Width)
- C: 通道数 (Channel) ， 如 RGB 为 3

而神经网络（尤其是 CNN）的输入通常要求 4 维张量 (N, C, H, W)：

- N: 批量大小 (Batch Size)
- C: 通道数
- H: 高度
- W: 宽度

因此需要 **两步操作**：

1. 增加批量维度: $(H, W, C) \rightarrow (1, H, W, C)$
 2. 调整轴顺序: $(1, H, W, C) \rightarrow (1, C, H, W)$
-

NumPy 实现

```
import numpy as np

img = np.random.rand(64, 64, 3)      # 模拟一张 (H=64, W=64, C=3) 的图片

# 方法一: np.expand_dims + np.transpose

img_4d = np.expand_dims(img, axis=0)    # (1, 64, 64, 3)
img_4d = np.transpose(img_4d, (0, 3, 1, 2)) # (1, 3, 64, 64)

# 方法二: np.newaxis + np.moveaxis

img_4d = img[np.newaxis, ...]           # (1, 64, 64, 3)
img_4d = np.moveaxis(img_4d, -1, 1)     # 把最后一维移到第1位 -> (1, 3, 64, 64)

# 方法三: reshape + transpose

img_4d = img.reshape(1, *img.shape)    # (1, 64, 64, 3)
img_4d = img_4d.transpose(0, 3, 1, 2)   # (1, 3, 64, 64)

# 方法四: 一行实现

img_4d = np.transpose(img[np.newaxis], (0, 3, 1, 2)) # (1, 3, 64, 64)
```

PyTorch 实现

```
import torch

img_t = torch.randn(64, 64, 3) # 模拟一张图片

# unsqueeze + permute

img_4d = img_t.unsqueeze(0) # (1, 64, 64, 3)

img_4d = img_t.permute(2, 0, 1) # 先转成 (C, H, W)

img_4d = img_4d.unsqueeze(0) # (1, C, H, W)

# 或者一行：

img_4d = img_t.unsqueeze(0).permute(0, 3, 1, 2) # (1, 3, 64, 64)

# view + transpose

img_4d = img_t.unsqueeze(0).transpose(1, 3).transpose(2, 3) # 两步交换
```

可微分张量与阻止计算图追踪

一、什么是可微分张量 (Differentiable Tensor)

可微分张量是自动微分 (Autograd) 机制的核心概念，以 PyTorch 为例：

- 当一个张量设置 `requires_grad=True` 时，它就是**可微分张量**
- PyTorch 会自动记录对该张量的所有操作，构建一个**动态计算图** (Computational Graph)
- 调用 `.backward()` 时，自动沿着计算图反向传播计算梯度

```
import torch

# 创建一个可微分张量

x = torch.tensor([2.0, 3.0], requires_grad=True) # x 是可微分的

y = x ** 2 + 3 * x # 计算图中的操作被记录

z = y.sum() # 标量输出

z.backward() # 反向传播

print(x.grad) # 梯度:  $dy/dx = 2x + 3 \rightarrow [7.0, 9.0]$ 
```

计算图结构:

```
x(requires_grad=True) → x^2 + 3x → y → sum → z(标量) → backward()
```

二、阻止计算图追踪的几种方式

在实际应用中，有时不需要计算梯度（如模型推理、参数更新等），可以通过以下方式阻止追踪：

方式一： `torch.no_grad()`（最常用）

上下文管理器，在其包裹的代码块中临时禁用梯度计算。


```

x = torch.tensor([2.0, 3.0], requires_grad=True)

with torch.no_grad():          # 块内所有操作都不会被追踪

    y = x * 2                  # 不会构建计算图

    print(y.requires_grad)    # False

# 离开块后恢复

z = x * 2

print(z.requires_grad)        # True

```

适用场景：模型推理（eval）、测试阶段、参数更新时

方式二： `.detach()`

从当前计算图中**分离**出一个新的张量，新张量与原张量共享数据，但不再关联计算图。

```

x = torch.tensor([2.0, 3.0], requires_grad=True)

y = x ** 2

y_detached = y.detach()      # 分离，y_detached 与 y 共享数据但不追踪

print(y_detached.requires_grad) # False

# 分离后可以继续操作，但不会影响原计算图

z = y_detached * 2           # 不会追踪到 x

```

适用场景：只取中间结果的值但不希望影响梯度回传

方式三： `.requires_grad_(False)`

原地修改张量的 `requires_grad` 属性，关闭可微分性。

```
x = torch.tensor([2.0, 3.0], requires_grad=True)

x.requires_grad_(False)      # 原地关闭，之后所有操作都不追踪

print(x.requires_grad)       # False
```

适用场景：确定某个张量后续都不需要梯度

方式四： `torch.set_grad_enabled(bool)`

全局开关，控制是否启用梯度计算。

```
# 关闭梯度

torch.set_grad_enabled(False)

x = torch.tensor([2.0, 3.0], requires_grad=True)

print(x.requires_grad)       # True（属性不变），但操作不会被追踪

# 重新开启

torch.set_grad_enabled(True)
```

适用场景：需要灵活切换训练/评估模式时

方式五： `.eval()` 模式（间接影响）

将模型设为评估模式，配合某些层（如 Dropout、BatchNorm）的行为改变，不直接阻止追踪，但常与 `torch.no_grad()` 搭配使用。

```
model.eval()                  # 改变 Dropout/BatchNorm 等层的行为

with torch.no_grad():         # 禁用梯度追踪

    output = model(input)
```

期望、方差和标准差

期望是随机变量所有可能取值的**加权平均**，权重为对应的概率。

方差衡量随机变量取值与其期望之间的**偏离程度**（离散程度）。

标准差是方差的**算术平方根**，与原始数据具有相同的量纲。

交叉熵与交叉熵损失函数

一、什么是交叉熵（Cross-Entropy）

交叉熵来源于信息论，衡量两个概率分布 P 和 Q 之间的**差异**。

$$H(P, Q) = - \sum_x P(x) \log Q(x)$$

- P : 真实分布 (true label)
- Q : 预测分布 (model output)
- 交叉熵越小 $\rightarrow$ 两个分布越接近 $\rightarrow$ 模型预测越准

与 KL 散度的关系:

$$H(P, Q) = H(P) + D_{KL}(P \parallel Q)$$

交叉熵 = 真实分布的熵 + KL 散度。当 P 固定时，最小化交叉熵等价于最小化 KL 散度。

二、常用的交叉熵损失函数

1. 二分类交叉熵损失 (Binary Cross-Entropy, BCE)

用于**二分类**问题（如判断是猫还是狗）。

$$\mathcal{L}_{BCE} = -\frac{1}{N} \sum_{i=1}^N [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]$$

- $y_i \in \{0, 1\}$: 真实标签 (0 或 1)
- $\hat{y}_i \in (0, 1)$: 模型预测的概率 (经 Sigmoid 输出)

```
import torch

import torch.nn as nn

# BCE 示例

loss_bce = nn.BCELoss()

pred = torch.sigmoid(torch.randn(4))          # 模拟预测概率

target = torch.tensor([1.0, 0.0, 1.0, 0.0])    # 真实标签

print(loss_bce(pred, target))
```

```
# BCEWithLogitsLoss = Sigmoid + BCELoss (数值更稳定)

loss_bce_logits = nn.BCEWithLogitsLoss()

logits = torch.randn(4)                       # 未经过 Sigmoid 的原始输出

print(loss_bce_logits(logits, target))
```

2. 多分类交叉熵损失 (Categorical Cross-Entropy, CCE)

用于**多分类**问题（如手写数字识别 0~9）。

$$\mathcal{L}_{CCE} = -\frac{1}{N} \sum_{i=1}^N \sum_{c=1}^C y_{i,c} \log(\hat{y}_{i,c})$$

- C : 类别总数
- $y_{i,c}$: one-hot 编码的真实标签
- $\hat{y}_{i,c}$: Softmax 输出的概率分布

```
# CCE 示例

loss_cce = nn.CrossEntropyLoss()              # 内置 Softmax

logits = torch.randn(4, 10)                   # 4个样本, 10个类别

targets = torch.tensor([2, 7, 1, 9])          # 类别索引 (无需 one-hot)

print(loss_cce(logits, targets))
```

3. 带权重的交叉熵损失 (Weighted Cross-Entropy)

给不同类别加上权重，解决**类别不平衡**问题。

$$\mathcal{L}_{WCE} = -\frac{1}{N} \sum_{i=1}^N w_{y_i} \cdot \log(\hat{y}_{i,y_i})$$

```
# 加权示例：类别 0 权重 1.0，类别 1 权重 5.0（少数类）  
  
weights = torch.tensor([1.0, 5.0])  
  
loss_weighted = nn.CrossEntropyLoss(weight=weights)
```

4. 带标签平滑的交叉熵损失 (Label Smoothing)

不让模型对正确标签过于"自信"，防止过拟合。

- 将 one-hot 标签 $[0, 1, 0]$ 变为 $[\frac{\epsilon}{C-1}, 1 - \epsilon, \frac{\epsilon}{C-1}]$
- ϵ 通常取 0.1

```
# PyTorch 内置标签平滑（需较新版本）  
  
loss_smooth = nn.CrossEntropyLoss(label_smoothing=0.1)
```

三、均方误差损失 (MSE / Mean Squared Error)

MSE 是最基础的回归损失函数，计算预测值与真实值之间**差的平方的均值**。

$$\mathcal{L}_{MSE} = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2$$

- 常用于**回归任务**（房价预测、温度预测等）
- 也可用于分类，但效果通常不如交叉熵

四、直观理解

```
真实分布 P: [0, 1, 0, 0]      # 类别 1 是正确答案  
  
预测分布 Q1: [0.05, 0.90, 0.03, 0.02]  → 交叉熵小  ✓  
  
预测分布 Q2: [0.30, 0.40, 0.20, 0.10]  → 交叉熵大  ✗
```

关键点：

- 交叉熵关注**正确类别的预测概率** — 正确类别的概率越高，损失越低
- 对错误分类的**惩罚是对数级的** — 如果模型对正确类别给出很低概率，损失会非常大
- 相比均方误差（MSE），交叉熵在分类任务中**梯度更友好**，收敛更快

SQL

```
INSERT INTO person (id, name, sex) VALUES (1003, '小李子', 1);
```

```
DELETE FROM person WHERE name LIKE '%花%';
```

```
UPDATE person SET sex = 1 WHERE id = 1002;
```

```
SELECT sex, COUNT(*) AS 人数  
FROM person  
GROUP BY sex;
```

```
SELECT p.id, p.name, s.name AS sex  
FROM person p  
JOIN sex s ON p.sex = s.id;
```

第八题

```

import numpy as np
import json

class FaceFeatDetect:
    def __init__(self, path, feat_path):
        self.target_feat = self.load_target_feat(path)
        self.names, self.feats = self.load_feats(feat_path)

    def load_target_feat(self, path):
        with open(path, 'r', encoding = 'utf-8') as file:
            line = file.readline()
            tgt_lst = json.loads(line)
        return np.array(tgt_lst)

    def load_feats(self, path):
        with open(path, 'r', encoding = 'utf-8') as file:
            lines = file.readlines()
        names = []
        feats = []
        for line in lines:
            infos = line.strip().split("|")
            name = infos[1]
            feat_lst = json.loads(infos[2])
            names.append(name)
            feats.append(feat_lst)
        return names, np.array(feats)

    def call_similarity(self, top_k=3):
        dists = np.linalg.norm(self.feats - self.target_feat, axis=1)
        idx = np.argsort(dists)

        print(f"📌 前{top_k}个最相似特征（欧氏距离）:")
        print(f"{'排名':<6}{'姓名':<10}{'距离':<12}{'原索引'}")

        for rank, i in enumerate(idx[:top_k], 1):
            print(f"{rank:<6}{self.names[i]:<10}{dists[i]:.6f} (id={i})")

if __name__ == "__main__":
    path = "file/target_feat.txt"
    feat_path = "file/feat.txt"

    face = FaceFeatDetect(path, feat_path)
    print(f"目标特征维度: {face.target_feat.shape}")
    print(f"特征库总数: {len(face.names)}")
    face.call_similarity(top_k=3)

```

目标特征维度：(128,)

特征库总数：11



前3个最相似特征（欧氏距离）：

排名	姓名	距离	原索引
1	XY	0.000000	(id=10)
2	XY	0.661405	(id=9)
3	XY	0.734417	(id=8)

第九题

#第三题

```
import numpy as np
```

```
def compute_iou(box, boxes):
```

```
    x1_int = np.maximum(box[0], boxes[:,0])
```

```
    y1_int = np.maximum(box[1], boxes[:,1])
```

```
    x2_int = np.minimum(box[2], boxes[:,2])
```

```
    y2_int = np.minimum(box[3], boxes[:,3])
```

```
    int_w = np.maximum(0, x2_int - x1_int)
```

```
    int_h = np.maximum(0, y2_int - y1_int)
```

```
    int_area = int_w * int_h
```

```
    box_area = (box[2] - box[0]) * (box[3] - box[1])
```

```
    boxes_area = (boxes[:, 2] - boxes[:, 0]) * (boxes[:, 3] - boxes[:, 1])
```

```
    union_area = box_area + boxes_area - int_area
```

```
    iou = np.where(union_area > 0, int_area / union_area, 0.0)
```

```
    return iou
```

```
box = np.array([20, 20, 60, 60])
```

```
boxes = np.array([[30, 30, 70, 70],  
                  [30, 30, 50, 50],  
                  [70, 70, 90, 90],  
                  [60, 20, 80, 60]])
```

```
ious = compute_iou(box, boxes)
```

```
for i, iou in enumerate(ious):
```

```
    print(f"boxes[{i}] 的 IOU = {iou:.4f}")
```


boxes[0] 的 IOU = 0.3913

boxes[1] 的 IOU = 0.2500

boxes[2] 的 IOU = 0.0000

boxes[3] 的 IOU = 0.0000